

Compute4Me Study Notes — Section 3: Ray and General-Purpose AI Compute Engines

Context: This doc explains Section 3 of the Compute4Me research report. The focus is Ray — a general-purpose distributed compute engine that already realizes many ideas similar to what Compute4Me wants: a unified way to scale Python/AI workloads from a laptop to a cluster with dynamic scheduling and built-in ML libraries. This doc breaks down what Ray is, how it's implemented, how hard it is to use vs reimplement, and how it maps to your Compute4Me vision.

3.1 What Ray Actually Is

Ray is a **unified distributed computing framework** for Python, originally designed for emerging AI workloads like reinforcement learning and large-scale hyperparameter tuning.

Core references:

- **Ray paper (OSDI 2018):** "Ray: A Distributed Framework for Emerging AI Applications" (Moritz et al.)
arXiv: <https://arxiv.org/abs/1712.05889>
USENIX page: <https://www.usenix.org/conference/osdi18/presentation/moritz>
- **Ray GitHub:** <https://github.com/ray-project/ray>
- **Ray docs/site:** <https://docs.ray.io> and <https://ray.io>

High-level goals

Ray's design goals line up closely with what you described for Compute4Me:

- **Unified runtime:** One engine that can express different kinds of parallelism: task-parallel, actor-based, streaming, RL, hyperparameter tuning, distributed training.[web:71][web:27][web:9]
- **Dynamic execution:** Can handle workloads where the computation graph is not static (e.g., RL where episodes generate new tasks on the fly).[web:71][web:73]
- **Scale transparency:** The same Python code can run on a laptop or on a cluster with minimal changes (mostly just `ray.init()` pointing at a cluster).[web:9][web:74]
- **Built-in ML libraries:** Ray provides higher-level libraries on top of the core runtime: RLlib (RL), Tune (HPO), Train (distributed training), Serve (model serving), etc.[web:9][web:79]

In other words, Ray already implements the idea of an "AI compute engine" very similar to your mental model for Compute4Me.

3.1.1 Programming Model: Tasks and Actors

Ray exposes two core primitives in Python:

1. Remote functions (tasks)
2. Remote classes (actors)

You decorate your Python functions and classes with `@ray.remote` to turn them into distributed units of execution.

Remote functions (stateless tasks)

```
import ray

ray.init() # connect to local or remote Ray cluster

@ray.remote
def f(x):
    return x * x

# This call executes f(x) as a remote task on the cluster
future = f.remote(2)
result = ray.get(future) # -> 4
```

Conceptually:

- `f.remote(2)` submits a task to the Ray cluster.
- The Ray scheduler decides *where* to run it.
- The return value is an object reference (a future) that you can `ray.get()` later to fetch the result.

You can spawn thousands to millions of these tasks; Ray handles distributing them across all available CPUs/GPUs.

Remote actors (stateful workers)

```
@ray.remote
class Counter:
    def __init__(self):
        self.value = 0
    def inc(self):
        self.value += 1
        return self.value

counter = Counter.remote()

fut1 = counter.inc.remote() # executed on some worker process
fut2 = counter.inc.remote()
print(ray.get(fut1), ray.get(fut2)) # -> 1, 2
```

Actors give you **stateful services** in the cluster — basically long-running Python objects bound to a specific worker process. This maps nicely to things like:

- Parameter servers
- Data loaders
- Long-lived model-serving processes

Why this matters for Compute4Me

For Compute4Me, these primitives provide:

- A clean way to represent "job shards" as tasks.
- A way to represent each **worker node** as an actor that the master can talk to.
- A natural API for users: decorate parts of their code with `@ray.remote` and let Compute4Me orchestrate where those tasks run.

Difficulty to use: Low-medium. From your perspective as a deep learning engineer, the API is very friendly — annotate functions/classes and call `.remote()`.

Difficulty to reimplement from scratch: High. You'd need to build:

- A distributed scheduler
- A distributed object store
- A global metadata store
- Fault tolerance mechanisms

Compute4Me does *not* want to rebuild this entire layer; it can either:

- Use Ray as the compute substrate and focus on DL-specific orchestration
- Or adopt a much simpler custom RPC pattern if you don't need Ray's full generality

3.1.2 Architecture: Ray's Distributed Runtime

The Ray paper describes a fairly sophisticated runtime.^{[web:71][web:27][web:73]}

Key components:

1. Global Control Store (GCS)
2. Distributed object store (Plasma)
3. Bottom-up scheduler (local + global schedulers)

Global Control Store (GCS)

GCS is Ray's metadata brain.

- Tracks all tasks, object references, actors, and their locations.
- Originally implemented on top of Redis; newer versions use internal sharded storage.
- Provides pub/sub functionality so components can subscribe to metadata changes.

For Compute4Me, this is analogous to your master node's **metadata database**:

- Which Docker workers are registered
- What resources they have (GPU count, VRAM, CPU cores)
- Which tasks are running where

Distributed Object Store

Ray stores large objects (model weights, tensors, datasets, intermediate results) in a **shared-memory object store** on each node.

- Backed by Apache Arrow's Plasma store in earlier versions.
- Objects are immutable — once created, not modified — which simplifies consistency and fault tolerance.
- If memory is exhausted, objects can be spilled to disk with LRU policies.[web:73]

This means tasks don't pass big tensors around by value; they pass **references** to objects stored in the object store.

For Compute4Me:

- You'll want something similar — maybe not as fancy as Plasma, but at least:
 - A way to keep large arrays on a node without re-sending them over the network.
 - A way to refer to objects by ID.

Scheduler: Local + Global (Bottom-up Scheduling)

Ray uses a **hierarchical scheduler**:[web:73]

- Each node runs a **local scheduler** that tries to schedule tasks on its own resources.
- When the local node is overloaded or doesn't have the needed resources, tasks are forwarded to a **global scheduler**.
- The global scheduler has a global view of cluster loads and can assign tasks to other nodes.

This is known as *bottom-up scheduling*: tasks are first sent locally, escalate only if necessary.

For Compute4Me:

- Your **master node** acts like Ray's global scheduler.

- Each worker container may also have a minischeduler for local GPUs/CPU.
- Later, you can experiment with more advanced scheduling strategies (e.g., heterogeneity-aware placement).

Difficulty to reimplement:

- A basic central scheduler (one master, N workers) is easy to write with Python + gRPC.
- Ray's scalable hierarchical scheduler is harder: requires careful design to avoid bottlenecks at high task rates.

3.2 Ray as an AI Compute Engine (Libraries on Top)

Ray doesn't stop at being a low-level runtime. It includes higher-level libraries for common ML patterns:

- Ray Train — distributed deep learning
- Ray Tune — hyperparameter and architecture search
- Ray RLlib — reinforcement learning
- Ray Serve — model serving / online inference

All of these run on the same underlying primitives (tasks, actors, object store).

3.2.1 Ray Train: Distributed Deep Learning

Ray Train is the library that does exactly what Compute4Me wants for DL jobs: scale PyTorch/TensorFlow/Horovod training across multiple nodes with minimal code changes. [web:36][web:77]

- Blog (2026): "Distributed Deep Learning with Ray Train is Now In Beta"
<https://www.anyscale.com/blog/distributed-deep-learning-with-ray-train-is-now-in-beta>

Key features:[web:36][web:77]

- Scale to multi-GPU and multi-node training with 0 or minimal code changes.
- Runs on any cluster: on-prem, AWS, GCP, Azure, Kubernetes.
- Supports PyTorch, TensorFlow, and Horovod backends.
- Integrates with Ray Data for distributed data loading.
- Includes logging to TensorBoard, MLflow, etc.

Example: PyTorch training with Ray Train

Rough flow (simplified):

```
from ray import train
from ray.train import Trainer
from ray.train.torch import TorchTrainer

# Define your training loop
```

```
def train_loop_per_worker(config):
    import torch
    import torch.nn as nn
    import torch.optim as optim
    from ray import train

    model = MyModel()
    model = train.torch.prepare_model(model) # wraps in DDP under the hood

    optimizer = optim.Adam(model.parameters(), lr=config["lr"])
    train_dataset = ... # sharded automatically by Ray

    for epoch in range(config["epochs"]):
        for batch in train_dataset:
            optimizer.zero_grad()
            loss = model(batch["x"]).loss
            loss.backward()
            optimizer.step()

            train.report({"loss": loss.item()})

trainer = TorchTrainer(
    train_loop_per_worker=train_loop_per_worker,
    scaling_config={"num_workers": 4, "use_gpu": True},
    run_config=...
)

result = trainer.fit()
```

Internally:

- Ray Train spins up 4 worker actors, each on a GPU.
- Wraps the model in DDP or Horovod automatically.
- Handles data sharding, checkpointing, and log aggregation.

Difficulty to use in an existing project: Low. For most PyTorch training loops, you:

- Wrap your model with `train.torch.prepare_model`.
- Use Ray's dataset abstractions.
- Move training logic into `train_loop_per_worker`.

Difficulty to reimplement from scratch: High; you'd be rebuilding:

- Distributed worker orchestration
- Integration with DDP/Horovod
- Data pipeline abstraction + sharding
- Logging/checkpointing infrastructure

For Compute4Me, you can absolutely:

- Use Ray Train directly as the training backend.

- Let your own master layer decide which hardware (nodes) join a `Trainer` cluster and how many workers to spawn.

3.2.2 Ray Tune: Hyperparameter and Architecture Search

Ray Tune is a scalable HPO library built on Ray: <https://docs.ray.io/en/latest/tune/index.html>

It supports:

- Grid, random, Bayesian, HyperBand, ASHA, PBT, etc.
- Scaling trials across many nodes.
- Early stopping, checkpointing, resume.

For Compute4Me:

- You can treat each trial as a "task" scheduled on one or more workers.
- Your heterogeneity-aware scheduler can decide which trials go to which nodes.

Difficulty:

- Using Ray Tune in your projects: low-medium.
- Rebuilding a full-featured HPO engine with all these algorithms: quite high.

3.2.3 Ray Serve: Model Serving

Ray Serve provides a scalable, programmable model-serving framework on top of Ray.

- Docs: <https://docs.ray.io/en/latest/serve/index.html>

It lets you:

- Define deployments as Python classes.
- Load models, handle HTTP requests, and scale replicas.

This is relevant to Compute4Me if you also want:

- **Distributed inference** on your volunteer cluster (e.g., large exoplanet inference jobs, or serving models closer to users).

Difficulty:

- Using Ray Serve: medium (understanding deployment patterns, scaling options).
- Reimplementing a robust serving layer with autoscaling and routing: high.

3.3 How Hard Is It to Build Ray-like Capabilities Yourself?

Breaking it down:

3.3.1 Implementing Ray in an Existing Project (Using It)

For existing PyTorch/TensorFlow projects, using Ray is usually:

- `pip install ray`
- Add `ray.init()` and `@ray.remote` to the pieces you want to run remotely.
- For Ray Train: move training logic into a per-worker loop, use `TorchTrainer` or `TensorflowTrainer`.

From your current skillset, this is very manageable. The heavy lifting is cluster setup:

- Running the Ray head node: `ray start --head --port=6379`
- Starting workers: `ray start --address='head_ip:6379'`

This is already very close to the "docker run to join the cluster" UX you want for Compute4Me.

3.3.2 Re-making Ray from Scratch

To reimplement Ray's core ideas yourself, you'd need to build:

1. RPC layer between master and workers (gRPC/ZeroMQ/custom HTTP).
2. Task queue and scheduler that assigns tasks to workers based on load/resources.
3. Object store or at least a system to manage large tensor blobs efficiently.
4. Metadata store (which tasks exist, which are done, where results live).
5. Fault-tolerance (retry tasks if workers die, reassign tasks, track lineage if needed).
6. Library APIs to make it usable (decorators, futures, convenient training utilities).

At that point you'd be reinventing a large subset of Ray.

As a single person, it's realistic to:

- Build a **simplified Ray** that does just what Compute4Me needs:
 - A master that knows available workers, their resources.
 - Workers that register themselves, run tasks (train/eval/infer), and return results.
 - Simple, robust protocols rather than a full general-purpose runtime.

But you probably don't want to re-create *all* of Ray's capabilities.

3.4 How Ray Relates to Compute4Me's Vision

Your Compute4Me concept:

- Any Ubuntu box runs a Docker container and becomes a worker.
- A master node orchestrates jobs, partitions workloads based on worker specs, aggregates results.
- Focus on deep learning workloads.

Ray's current reality:

- Any machine can run a Ray worker and join a Ray cluster.
- A head node runs the scheduler and GCS.
- Ray Train/Tune/Serve provide first-class support for DL, tuning, and serving.

Overlap:

- The "AI compute engine" idea is nearly identical.[web:9][web:71]
- Ray already has APIs to do what you want to offer users (task/actor abstraction, DL training, HPO, serving).

What Ray doesn't directly do (your potential research/engineering space):

1. Volunteer / opportunistic nodes UX:

- Ray assumes you control the cluster (on-prem, cloud, Kubernetes).
- You want a "download Docker, run, join someone else's cluster" volunteer model.

2. Deep-learning-specific, heterogeneity-aware scheduling:

- Ray's scheduler is generic; it isn't deeply tuned to hetero GPUs, VRAM, bandwidth, etc., in the way you might want for exoplanet workloads.

3. Security / trust / quotas for external workers:

- Ray assumes nodes are trusted and belong to the same admin domain.
- Compute4Me may have semi-trusted/unknown nodes joining.

Concrete directions using Ray:

- **Compute4Me built on Ray:**
 - Master = Ray head node wrapped with your own coordination/service layer.
 - Workers = Docker images that auto-run `ray start --address=...` on startup.
 - User interface = your custom CLI/web UI that manages Ray clusters and launches Ray Train jobs.
- **Compute4Me as a simpler custom engine:**
 - Use ideas from Ray but implement a minimal RPC/task system yourself.
 - Use PyTorch DDP/Horovod for gradient sync.
 - Implement just enough scheduling and object storage for your use cases.

From a "build vs buy" angle, Ray gives you a proven, open-source substrate with high-quality engineering. It's reasonable to:

- Start with Compute4Me-on-Ray to get a working system.
- Then iteratively decide whether any parts need to be replaced with custom logic (e.g., a more specialized scheduler or security layer).

3.5 Key Links and Repos Summary

Ray core and ecosystem

- Ray GitHub (core runtime + all libraries):
<https://github.com/ray-project/ray>
- Ray docs:
<https://docs.ray.io>
- Ray paper (OSDI 2018):
arXiv: <https://arxiv.org/abs/1712.05889>
USENIX: <https://www.usenix.org/conference/osdi18/presentation/moritz>

Ray Train (distributed DL)

- Blog (Ray Train beta):
<https://www.anyscale.com/blog/distributed-deep-learning-with-ray-train-is-now-in-beta>
- Distributed training overview (Anyscale):
<https://www.anyscale.com/blog/what-is-distributed-training>

Ray Tune (HPO)

- Docs:
<https://docs.ray.io/en/latest/tune/index.html>

Ray RLlib and Serve

- RLlib (RL):
<https://docs.ray.io/en/latest/rllib/index.html>
- Ray Serve (model serving):
<https://docs.ray.io/en/latest/serve/index.html>

Awesome Ray (ecosystem list)

- Curated list of Ray projects:
<https://github.com/JiahaoYao/awesome-ray>

Next: Section 4 — Elastic and Heterogeneous Distributed Training (EasyScale, resource scheduling, etc.), where we'll see how people handle changing GPU counts and heterogeneity in production.